

# Imbalanced Classification and Sparsity

## MATH 480 Written Report

Jiajun Zhang (Last Updated: Feb 5th, 2026)\*

February 6, 2026

### Contents

|            |                                                              |           |
|------------|--------------------------------------------------------------|-----------|
| <b>I</b>   | <b>An Overview of Classical Classification Techniques</b>    | <b>2</b>  |
| <b>1</b>   | <b>Linear Classifiers</b>                                    | <b>2</b>  |
| 1.1        | Linear Discriminant Analysis (LDA) . . . . .                 | 2         |
| 1.2        | Support Vector Machines (SVMs): Separable Case . . . . .     | 6         |
| <b>2</b>   | <b>Non-linear Classifiers</b>                                | <b>7</b>  |
| 2.1        | Support Vector Machines (SVMs): Non-separable case . . . . . | 7         |
| 2.2        | $k$ th Nearest Neighbor (KNN) . . . . .                      | 15        |
| 2.3        | Logistic Regression . . . . .                                | 16        |
| <b>3</b>   | <b>Tree-based Classification Algorithms</b>                  | <b>18</b> |
| 3.1        | Decision Trees: An Introduction . . . . .                    | 18        |
| 3.2        | Decision Tree Algorithms . . . . .                           | 20        |
| 3.3        | Random Forests . . . . .                                     | 25        |
| <b>II</b>  | <b>Techniques for Analyzing Data</b>                         | <b>26</b> |
| <b>4</b>   | <b>Dimension Reduction</b>                                   | <b>26</b> |
| 4.1        | Fisher's Discriminant Analysis . . . . .                     | 26        |
| 4.2        | Principle Component Analysis . . . . .                       | 29        |
| <b>III</b> | <b>References</b>                                            | <b>30</b> |
| <b>5</b>   | <b>Bibliography</b>                                          | <b>30</b> |

---

\*Department of Mathematics and Statistics, jiajun.zhang@mail.mcgill.ca, McGill ID: 261105766

## Part I

# An Overview of Classical Classification Techniques

## 1 Linear Classifiers

### 1.1 Linear Discriminant Analysis (LDA)

The idea of linear discriminant analysis is to assume the conditional distribution of data  $\mathbf{x} \in \mathbb{R}^p$  given the class label  $y_i = c$ ,  $c = 1, \dots, k$  forms a multivariate normal distribution with mean  $\boldsymbol{\mu}_c$  and common covariance matrix  $\boldsymbol{\Sigma}$ . That is, we assume

$$\mathbf{X} = \mathbf{x} | Y = c \sim f_c(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} \cdot |\boldsymbol{\Sigma}|^{1/2}} \exp \left\{ -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) \right\}.$$

We may further assume a prior distribution on  $p_c = \mathbb{P}(Y = c)$ ,  $c = 1, \dots, k$ . Then by Bayes theorem, the posterior distribution,  $\mathbb{P}(Y = c | \mathbf{X} = \mathbf{x})$ , will satisfy

$$\mathbb{P}(Y = c | \mathbf{X} = \mathbf{x}) \propto p_c \cdot f_c(\mathbf{x}).$$

If we assume fixed-design and homoskedasticity, the covariance matrix  $\boldsymbol{\Sigma}$  will be constant, and further we may write the posterior probability as

$$\mathbb{P}(Y = c | \mathbf{X} = \mathbf{x}) = C p_c \cdot \exp \left\{ -\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) \right\}, \quad (1.1)$$

for some constant  $C$ . The log posterior probability of (1.1) is then given by

$$\begin{aligned} \log \mathbb{P}(Y = c | \mathbf{X} = \mathbf{x}) &= \log C + \log p_c - \frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu}_c) \\ &= \log C + \log p_c - \frac{1}{2} \left[ \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \mathbf{x} - 2\mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c \right] \\ &= C' + \log p_c - \frac{1}{2} \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c, \end{aligned}$$

where  $C'$  is a new constant that contains all the terms independent of the class  $c$ . From here, we define the *discriminant functions* as

$$\delta_c(\mathbf{x}) = \log p_c - \frac{1}{2} \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_c, \quad c = 1, \dots, k. \quad (1.2)$$

Hence the classification rule is simply compute each  $\delta_c(\mathbf{x})$  and assign  $Y$  the label  $c'$  such that  $\delta_{c'}(\mathbf{x})$  is maximized, i.e.,

$$\hat{Y} = \arg \max_c \delta_c(\mathbf{x}) := \arg \max_c \mathbb{P}(Y = c | \mathbf{X} = \mathbf{x}).$$

The *decision boundary* is the set of points  $\mathbf{x}$  such that the discriminant functions of two classes agree at  $\mathbf{x}$ . With  $k$  classes, we are able to get  $\binom{k}{2}$  different decision boundaries. For example, the decision boundary between class  $l$  and  $m$  are given by the linear equation  $\delta_l(\mathbf{x}) = \delta_m(\mathbf{x})$ :

$$\log p_l - \frac{1}{2} \boldsymbol{\mu}_l^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_l + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_l = \log p_m - \frac{1}{2} \boldsymbol{\mu}_m^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_m + \mathbf{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_m. \quad (1.3)$$

Then the solution to (1.3) is given by

$$(\boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_l - \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_m)^\top \mathbf{x} = \log \frac{p_m}{p_l} - \frac{1}{2} (\boldsymbol{\mu}_m^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_m - \boldsymbol{\mu}_l^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{\mu}_l).$$

In practice, we often do not have access to the true probability density  $f_c(\mathbf{x})$  and  $p_c(\mathbf{x})$ . We would replace them by their estimates based on the data we have. The in-class mean  $\boldsymbol{\mu}_c$ , prior probability  $p_c(\mathbf{x})$ , and the covariance matrix  $\boldsymbol{\Sigma}$  can be estimated by

$$\hat{\boldsymbol{\mu}}_c = \frac{1}{n_c} \sum_{i: y_i=c} \mathbf{x}_i, \quad \hat{p}_c(\mathbf{x}) = \frac{n_c}{n}, \quad \hat{\boldsymbol{\Sigma}} = \frac{1}{n-k} \sum_{c=1}^k \sum_{i: y_i=c} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c) \cdot (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)^\top,$$

hence the estimated decision boundary in (1.3) can be written as  $\hat{\delta}_l(\mathbf{x}) = \hat{\delta}_m(\mathbf{x})$ :

$$(\hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_l - \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_m)^\top \mathbf{x} = \log \frac{\hat{p}_m}{\hat{p}_l} - \frac{1}{2} (\hat{\boldsymbol{\mu}}_m^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_m - \hat{\boldsymbol{\mu}}_l^\top \hat{\boldsymbol{\Sigma}}^{-1} \hat{\boldsymbol{\mu}}_l),$$

which takes the form of a linear function

$$\mathbf{w}^\top \mathbf{x} + b = 0.$$

Apart from linear discriminant analysis, quadratic discriminant analysis (QDA) is also widely used. The set up is similar to LDA, however we assume heteroskedasticity where the covariance matrix depends on the class label  $c$ :

$$\mathbf{X} = \mathbf{x} | Y = c \sim f_c(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} \cdot |\boldsymbol{\Sigma}_c|^{1/2}} \exp \left\{ -\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_c)^\top \boldsymbol{\Sigma}_c^{-1} (\mathbf{x} - \boldsymbol{\mu}_c) \right\}.$$

Then the discriminant function is quadratic:

$$\begin{aligned} \delta_c(\mathbf{x}) &= \log p_c - \frac{1}{2} \mathbf{x}^\top \boldsymbol{\Sigma}_c^{-1} \mathbf{x} + \mathbf{x}^\top \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c - \frac{1}{2} \boldsymbol{\mu}_c^\top \boldsymbol{\Sigma}_c^{-1} \boldsymbol{\mu}_c \\ &= \mathbf{x}^\top \mathbf{A} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + c. \end{aligned}$$

The within-class covariance matrix can be estimated by

$$\hat{\boldsymbol{\Sigma}}_c^{-1} = \frac{1}{n_c - 1} \sum_{i: y_i=c} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c) \cdot (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_c)^\top,$$

Finally we have the estimated decision boundary  $\hat{\delta}_l(\mathbf{x}) = \hat{\delta}_m(\mathbf{x})$  between class  $l$  and  $m$  given by

$$\frac{1}{2} \mathbf{x}^\top (\hat{\boldsymbol{\Sigma}}_l^{-1} - \hat{\boldsymbol{\Sigma}}_m^{-1}) \mathbf{x} + \mathbf{x}^\top \hat{\boldsymbol{\Sigma}}_m^{-1} \hat{\boldsymbol{\mu}}_m - \mathbf{x}^\top \hat{\boldsymbol{\Sigma}}_l^{-1} \hat{\boldsymbol{\mu}}_l - \frac{1}{2} \hat{\boldsymbol{\mu}}_m^\top \hat{\boldsymbol{\Sigma}}_m^{-1} \hat{\boldsymbol{\mu}}_m + \frac{1}{2} \hat{\boldsymbol{\mu}}_l^\top \hat{\boldsymbol{\Sigma}}_l^{-1} \hat{\boldsymbol{\mu}}_l + \log \frac{\hat{p}_m}{\hat{p}_l} = 0.$$

We next present a study on wine dataset from **R**. These data are the results of a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars (labeled cultivar 1 (lack), 2 (pink), 3 (green) in Figure 1 shown on the next page). The analysis determined the quantities of 13 constituents found in each of the three types of wines. We perform LDA and QDA based on six groups of binary covariates: Alcohol (V1) and Malic Acid (V2); Ash (V3) and Total phenols (V6); Alcalinity of ash (V4) and Nonflavanoid phenols (V8); Magnesium (V5) and Flavanoids (V7); Proanthocyanins (V9) and Color intensity (V10), Hue (V11) and Proline (V13). The data points are plotted in Figure 1, where different colors refer to different classes (cultivars). The LDA decision boundaries are shown in black, and the QDA decision boundaries are shown in red.

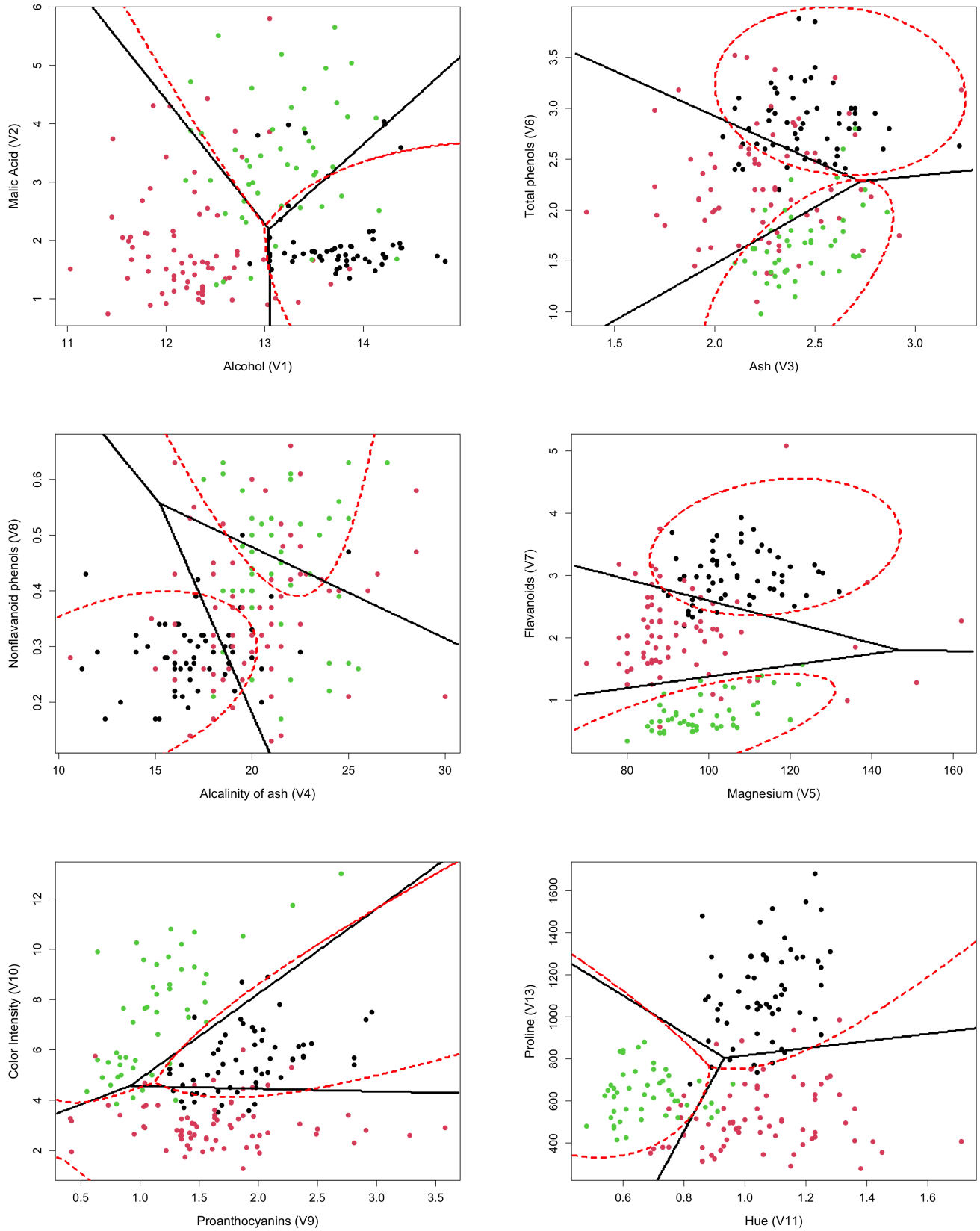


Figure 1: The decision boundaries of LDA and QDA based on different groups of binary co-variates of wines from three cultivars.

## 1.2 Support Vector Machines (SVMs): Separable Case

We again consider a binary classification problem with data  $\mathcal{D}_n := \{(x_1, y_1), \dots, (x_n, y_n)\}$ , where  $x_i$  is the  $p$ -dimensional covariates and  $y_i \in \{-1, 1\}$  is binary. Support vector machines (SVMs) is another linear classifier used for classification, but the development of SVMs is different from LDA. Unlike LDA, SVMs do not assume any prior distribution on the data, but instead try to find the optimal  $p$ -dimensional vector  $w$  and scalar  $b$  such that

$$\begin{cases} x^\top w + b > 0 & \text{if } y = 1 \\ x^\top w + b < 0 & \text{if } y = -1 \end{cases}$$

and the decision boundary is given by the equation  $x^\top w + b = 0$ . In many cases, we are able to find multiple linear functions that perfectly separate the two classes. Apart from separability, we also want to maximize the closest distance to the linear function in each class, so we are able to make more accurate predictions for data points near the decision boundary. See the two examples shown in Figure 2

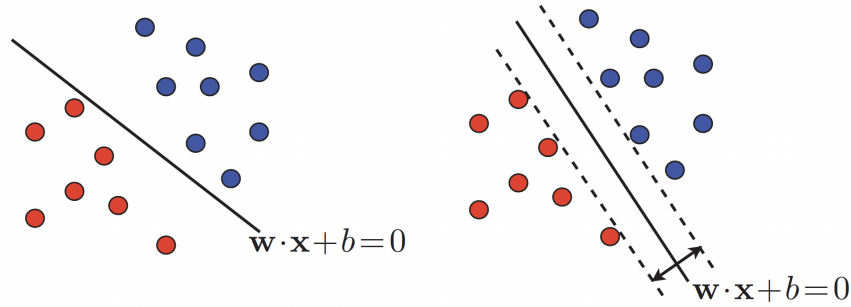


Figure 2: With the data set given, two linear functions are proposed. Both classifies the data well, but the one on the right has maximal separability. The figure is taken from Mohri et. al. [MRT12]

For any point  $(x_i, y_i) \in \mathcal{D}_n$ , it is easy to see that the distance of  $x_i$  to the linear function  $x^\top w + b = 0$  is given by

$$d_{x_i} = \frac{|x_i^\top w + b|}{\|w\|}.$$

For computational purposes, we may scale  $w, b$  such that  $\min_{(x_i, y_i) \in \mathcal{D}_n} |x_i^\top w + b| = 1$ . Hence the shortest distance from a data point to  $x^\top w + b = 0$  is simply 1. So for any data  $x_i$ , we always have  $|x_i^\top w + b| \geq 1$ , and the minimum distance is given by  $d_{\min} = 1/\|w\|$ . The *margin* (shortest distance between data from two different classes) is just  $2/\|w\|$ . Then the SVM finds the optimal classifier such that the margin is maximized while correctly classifying all the given data. i.e., we solve the optimization problem

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to } y_i \cdot (x_i^\top w + b) \geq 1, \quad (1.4)$$

for all  $i = 1, \dots, n$ . We transform (1.4) to its Lagrangian:

$$\mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i \cdot (x_i^\top w + b) - 1], \quad (1.5)$$

where  $\alpha_i \geq 0, i = 1, \dots, n$  are the Lagrange variables. The term (1.5) is not easy to optimize, so we usually introduce its dual problem given by

$$\mathcal{L}_{\text{dual}}(\alpha) = \max_{\alpha} \min_{w, b} \mathcal{L}(w, b, \alpha).$$

Note that the original optimization problem in (1.4) is strictly convex, so we can derive the unique minimizer of  $w$  and  $b$  by taking partial derivatives of the equation (1.5) with respect to  $w, b$ :

$$\frac{\partial \mathcal{L}}{\partial w} = w - \sum_{i=1}^n \alpha_i y_i x_i = 0, \quad \frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \alpha_i y_i = 0,$$

and equate them to zero. Hence we have

$$w = \sum_{i=1}^n \alpha_i y_i x_i, \quad \sum_{i=1}^n \alpha_i y_i = 0. \quad (1.6)$$

We plug (1.6) into (1.5), and the dual problem is now given by

$$\mathcal{L}_{\text{dual}}(\alpha) = \frac{1}{2} \left\| \sum_{i=1}^n \alpha_i y_i x_i \right\|^2 - \sum_{i=1}^n \alpha_i \left[ y_i \cdot \left( x_i^\top \sum_{i=1}^n \alpha_i y_i x_i + \sum_{i=1}^n \alpha_i y_i \right) - 1 \right].$$

After simplification, the optimization problem now is simply

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^\top x_j), \quad (1.7)$$

subject to  $\alpha_i \geq 0 \wedge \sum_{i=1}^n \alpha_i y_i = 0$  for all  $i = 1, \dots, n$ . The optimization problem is now a quadratic programming (QP) problem, and it can be solved by sequential minimal optimization (SMO) introduced later. Thus the solution suggested by SVMs is given by

$$f(x) = x^\top w + b = \sum_{i=1}^n \alpha_i y_i (x_i^\top x) + b,$$

where  $b$  can be obtained via the support vector  $x_j$  such that  $x_j^\top w + b = y_j$ :

$$b = y_j - \sum_{i=1}^n \alpha_i y_i (x_i^\top x_j),$$

and it can be used directly to determine the hypothesis returned by SVMs, by comparing  $y_i$  and  $\text{sgn}(f(x_i))$ .

## 2 Non-linear Classifiers

### 2.1 Support Vector Machines (SVMs): Non-separable case

We now return to the binary classification problem using support vector machines discussed in Section 1.2. It is worth to note that separability is not always ideal, since in many cases we would have  $y_i \cdot (x_i^\top w + b) \not\geq 1$  for any linear function, as shown in Figure 3.

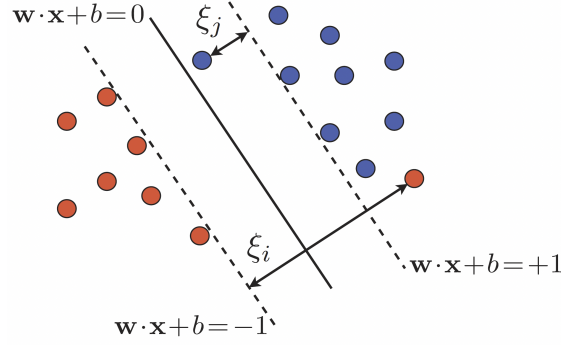


Figure 3: The case when the data can not be fully separated by a linear function. The figure is taken from Mohri et. al. [MRT12].

Under this situation, we may also want to minimize the “error” made by the linear function  $\xi_i$ , which measures the separation quantity of a data point from the decision boundary. The optimization problem in this case is given by

$$\min_{w, b, \xi_i} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i^p,$$

subject to  $y_i \cdot (x_i^\top w + b) \geq 1 - \xi_i$  and  $\xi_i \geq 0$ , where  $p \geq 1$  and  $C$  is a constant denoting the trade-off between margin maximization and error minimization. A common method to obtain  $C$  is by using  $n$ -fold cross validation. In fact Mohri et. al. (Section 4.3) [MRT12] showed that the optimization problem is identical to separable SVMs:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i^\top x_j), \quad (2.1)$$

subject to  $C \geq \alpha_i \geq 0 \wedge \sum_{i=1}^n \alpha_i y_i = 0$  for all  $i = 1, \dots, n$  and some constant  $C$ .

Kernel methods can be used to solve non-separable SVMs. The idea is to introduce a map  $\Phi : \mathcal{X} \rightarrow \mathcal{H}$  from the input data space  $\mathcal{X}$  to a higher dimensional space  $\mathcal{H}$  where separation can be done easily, and kernels are introduced to solve this problem. Briefly speaking, a kernel is a function  $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$  such that for any  $x, x' \in \mathcal{X}$ ,

$$K(x, x') = \langle \Phi(x), \Phi(x') \rangle, \quad (2.2)$$

where  $\langle \cdot, \cdot \rangle$  denotes the inner product on a Hilbert space  $\mathcal{H}$ . We say  $K$  is positive definite symmetric (PDS) if for any  $x_1, \dots, x_n \in \mathcal{X}$ , the gram matrix

$$K := \left[ K(x_i, x_j) \right]_{i,j} \in \mathbb{R}^{n \times n}$$

is symmetric positive semi-definite (SPSD). One property of PDS kernel is called reproducing kernel Hilbert space (RKHS), which is the idea of equation (2.2), i.e., the kernel function evaluated at points  $x, x'$  can be viewed as the inner product between  $\Phi(x)$  and  $\Phi(x')$  for our choice



of the map  $\Phi : \mathcal{X} \rightarrow \mathcal{H}$ . One choice of kernel is the polynomial kernel (of degree  $d$ ), which is defined by

$$K(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d,$$

for some constant  $c > 0$ . If we consider the classification problem in Figure 4 (left), we see that in  $\mathbb{R}^2$  it is non-separable. However if we apply a polynomial kernel of degree 2, we have

$$K(\mathbf{x}, \mathbf{x}') = (x_1 x'_1 + x_2 x'_2 + c)^2 = \begin{pmatrix} x_1^2 & x_2^2 & \sqrt{2}x_1 x_2 & \sqrt{2}c x_1 & \sqrt{2}c x_2 & c \end{pmatrix} \cdot \begin{pmatrix} x_1'^2 \\ x_2'^2 \\ \sqrt{2}x_1' x_2' \\ \sqrt{2}c x_1' \\ \sqrt{2}c x_2' \\ c \end{pmatrix},$$

and we project the transformed data into a 2-dimensional subspace (see Figure 4, right), we can easily see that the data can be classified by a hyperplane.

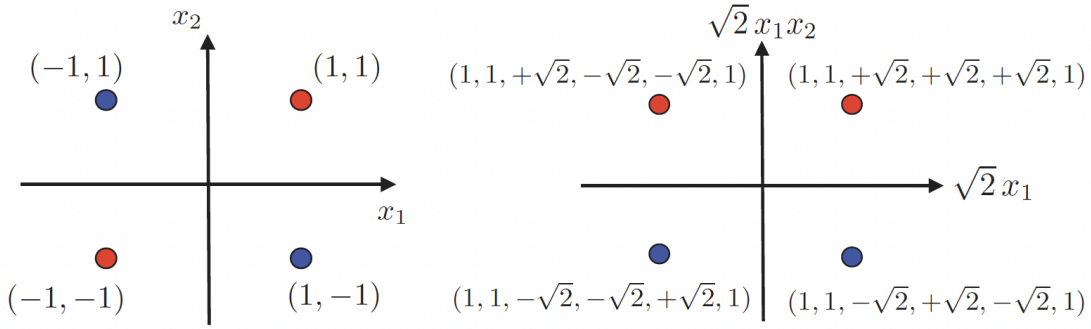


Figure 4: A binary XOR classification problem where linear separability is impossible in  $\mathbb{R}^2$  (Figure on the left). After applying a polynomial kernel the transformed data can be easily classified using a hyperplane (The figure on the right). The figure is taken from Mohri et. al. [MRT12].

Other choices of kernel functions include the Gaussian kernel:  $K(\mathbf{x}, \mathbf{x}') = \exp(-\|\mathbf{x} - \mathbf{x}'\|_2^2 / 2\sigma^2)$ , sigmoid kernel  $K(\mathbf{x}, \mathbf{x}') = \tanh(a(\mathbf{x}^\top \mathbf{x}') + b)$ , and many others. Once a specific kernel is chosen, we may replace the inner product term  $\mathbf{x}_i^\top \mathbf{x}_j$  appeared in (2.2) by a corresponding kernel estimate, which leads to the following optimization problem:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j). \quad (2.3)$$

Denote  $\mathbf{K}$  as the gram matrix, the optimization problem can also be written in the matrix form:

$$\max_{\alpha} 2\mathbf{1}^\top \cdot \alpha - (\alpha \circ \mathbf{y})^\top \mathbf{K}(\alpha \circ \mathbf{y}), \quad \text{subject to } \mathbf{0} \leq \alpha \leq \mathbf{C} \wedge \alpha^\top \mathbf{y} = 0.$$

The dual optimization problem is in fact a quadratic programming (QP) problem where specific algorithms have been developed. In practice, the *sequential minimal optimization* (SMO) proposed by Platt ([Pla98]) is often used due to its fast compiling time. Briefly speaking, SMO

is an iterative algorithm that breaks (2.3) down into a series of small-sized problems. We give a short overview of SMO algorithm, while a detailed implementation can be found in Platt [Pla98] as well as Mak [Mak00]. We first introduce the Karush-Kuhn-Tucker (KKT) condition commonly used for non-linear optimization problems. For each  $i = 1, \dots, n$ , if it satisfies

$$\begin{cases} \alpha_i = 0 & \iff y_i \cdot (\mathbf{x}_i^\top \mathbf{w} - b) \geq 1 \\ 0 < \alpha_i < C & \iff y_i \cdot (\mathbf{x}_i^\top \mathbf{w} - b) = 1, \\ \alpha_i = C & \iff y_i \cdot (\mathbf{x}_i^\top \mathbf{w} - b) \leq 1 \end{cases} \quad (2.4)$$

then the quadratic programming problem defined in (2.3) has been solved. We divide the SMO algorithm into four parts: ① The set up; ② Updating multipliers; ③ Updating  $b$ ; ④ Repeat until converge. A heuristic and simple derivation is given as follow.

① **The set up:** The SMO chooses to solve the smallest optimization problem at each step, and in our case the smallest possible problem is to solve the optimization problem involving two Lagrange multipliers  $\alpha_1, \alpha_2$  (without loss of generality, we shall denote  $\alpha_1, \alpha_2$  as the two Lagrange multipliers in each problem). The SMO then computes the constraints on these multipliers and solves for the constrained minimum. In each case, we have

$$y_1 \alpha_1 + y_2 \alpha_2 = k, \quad \text{subject to } 0 \leq \alpha_1, \alpha_2 \leq C \quad (2.5)$$

where  $\gamma$  is assumed to be a constant given by  $k := -\sum_{i=3}^n y_i \alpha_i$ . Since  $y_1, y_2 = \pm 1$ , so this 2D optimization problem is straightforward. The figure below illustrates the two cases with  $y_1, y_2$  having the same and opposite sign:

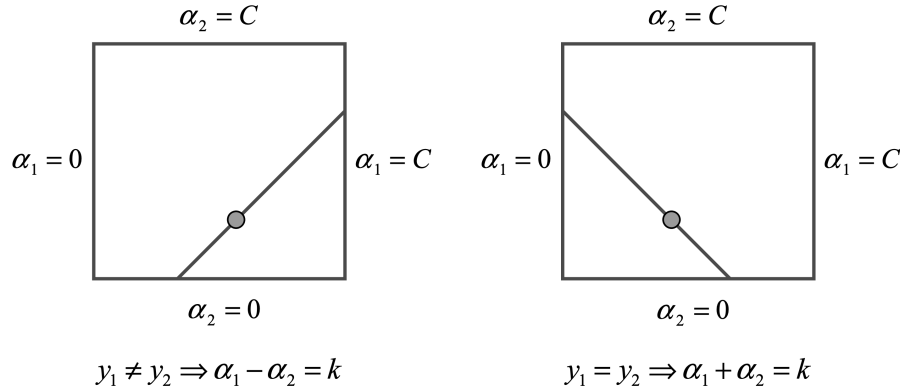


Figure 5: The 2D geometric view of the optimization problem defined in (2.5). The box represents the boundary conditions  $0 \leq \alpha_1, \alpha_2 \leq C$ , and the diagonal segment is given by the equation  $y_1 \alpha_1 + y_2 \alpha_2 = k$  with slope  $\pm 1$  depending on  $\text{sgn}(y_1 y_2)$ . The figure is taken from Platt [Pla98].

From Figure 5, we can see that the equation  $y_1 \alpha_1 + y_2 \alpha_2 = k$  will intercept the constraint box, we denote the intercept on horizontal axis as  $L$  and the intercept on vertical axis as  $H$ , where we restrict  $0 \leq L, H \leq C$ . Then it is easy to see that

$$\begin{cases} L = \max\{0, \alpha_2 - \alpha_1\}, H = \min\{C, C + \alpha_2 - \alpha_1\}, & \text{if } \text{sgn}(y_1 y_2) = -1 \\ L = \max\{0, \alpha_2 + \alpha_1 - C\}, H = \min\{C, \alpha_2 + \alpha_1\}, & \text{if } \text{sgn}(y_1 y_2) = 1 \end{cases}.$$

② **Updating  $\alpha_1, \alpha_2$ :** The objective function given by (2.3) with two multipliers  $\alpha_1, \alpha_2$  is given by

$$\mathcal{F}_{1,2} = \alpha_1 + \alpha_2 - \frac{1}{2} \left( \alpha_1^2 K(x_1, x_1) + 2\alpha_1 \alpha_2 y_1 y_2 K(x_1, x_2) + \alpha_2^2 K(x_2, x_2) \right),$$

since  $y_i^2 = 1, i = 1, 2$  and  $K(\cdot, \cdot)$  is symmetric. If we substitute  $\alpha_1$  using  $\alpha_1 = (k - y_2 \alpha_2)/y_1$  defined through (2.5), we have

$$\begin{aligned} \mathcal{F}_{1,2}(\alpha_2) = & \left( K(x_1, x_2) - \frac{1}{2} K(x_1, x_1) - \frac{1}{2} K(x_2, x_2) \right) \alpha_2^2 + \left( k y_2 K(x_1, x_1) - k y_2 K(x_1, x_2) - \frac{y_2}{y_1} + 1 \right) \alpha_2 \\ & + \frac{k}{y_1} + \frac{1}{2} k^2 K(x_1, x_1). \end{aligned} \quad (2.6)$$

To optimize (2.6), we may use the Newton's method to find the critical points of  $\mathcal{F}_{1,2}(\alpha_2)$  and update the new multiplier  $\alpha_2^{\text{new}}$  as

$$\alpha_2^{\text{new}} = \alpha_2 - \frac{\mathcal{F}'_{1,2}(\alpha_2)}{\mathcal{F}''_{1,2}(\alpha_2)}.$$

The first derivative of (2.6) is given by

$$\frac{\partial \mathcal{F}_{1,2}(\alpha_2)}{\partial \alpha_2} = 2\alpha_2 \cdot \left( K(x_1, x_2) - \frac{1}{2} K(x_1, x_1) - \frac{1}{2} K(x_2, x_2) \right) + k y_2 K(x_1, x_1) - k y_2 K(x_1, x_2) - \frac{y_2}{y_1} + 1.$$

We define the predicted value  $u_i$  and the error  $e_i$  as

$$u_i = f(x_i) = \sum_j \alpha_j y_j K(x_j, x_i) + b; \quad e_i = u_i - y_i,$$

then it can be shown that  $\mathcal{F}'_{1,2} = y_2(e_1 - e_2)$ . Meanwhile the second derivative of (2.6) is given by

$$\frac{\partial^2 \mathcal{F}_{1,2}(\alpha_2)}{\partial \alpha_2^2} = - (K(x_1, x_1) + K(x_2, x_2) - 2K(x_1, x_2)) := -\eta.$$

Hence the updated multiplier  $\alpha_2^{\text{new}}$  is given by

$$\alpha_2^{\text{new}} = \alpha_2 + \frac{y_2(e_1 - e_2)}{\eta}. \quad (2.7)$$

To insure the updated multiplier lies within the desired constraints, we define the clipped  $\alpha_2^{\text{new}}$  as

$$\alpha_2^{\text{new,clipped}} = \begin{cases} H & \text{if } \alpha_2^{\text{new}} > H \\ \alpha_2^{\text{new}} & \text{if } L < \alpha_2^{\text{new}} < H \\ L & \text{if } \alpha_2^{\text{new}} \leq L \end{cases}.$$

Note that as we update  $\alpha_2$ , the other multiplier  $\alpha_1$  will change as well. The updated  $\alpha_1$  can be directly computed by

$$\alpha_1^{\text{new}} = \alpha_1 + \text{sgn}(y_1 \cdot y_2) \cdot (\alpha_2 - \alpha_2^{\text{new,clipped}}),$$

using the constraint that

$$y_1\alpha_1 + y_2\alpha_2 = k \quad \text{and} \quad y_1\alpha_1^{\text{new}} + y_2\alpha_2^{\text{new,clipped}} = k.$$

③ Updating  $b$ : The decision function is given by

$$f(\mathbf{x}) = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b,$$

hence once we have obtained  $\alpha_2^{\text{new,clipped}}$ ;  $\alpha_1^{\text{new}}$ , the parameter  $b$  needs to be updated accordingly in order to fulfill the KKT conditions for both multipliers. If  $\mathbf{x}_i$  is a support vector, then we have  $y_i f(\mathbf{x}_i) = 1$  and furthermore

$$b = y_i - \sum_{j=1}^n \alpha_j y_j K(\mathbf{x}_j, \mathbf{x}_i).$$

We now replace  $\alpha_1, \alpha_2$  by their updates. and hence we have

$$f^{\text{new}}(\mathbf{x}_1) = f(\mathbf{x}_1) + (\alpha_1^{\text{new}} - \alpha_1) y_1 K(\mathbf{x}_1, \mathbf{x}_1) + (\alpha_2^{\text{new,clipped}} - \alpha_2) y_2 K(\mathbf{x}_2, \mathbf{x}_1) + (b^{\text{new}} - b).$$

Define  $e_1 = f(\mathbf{x}_1) - y_1$ , the updated  $b_1$  is given by

$$b_1 = b - e_1 - y_1(\alpha_1^{\text{new}} - \alpha_1) K(\mathbf{x}_1, \mathbf{x}_1) - y_2(\alpha_2^{\text{new,clipped}} - \alpha_2) K(\mathbf{x}_2, \mathbf{x}_1),$$

and the same idea applies for  $b_2$ :

$$b_2 = b - e_2 - y_1(\alpha_1^{\text{new}} - \alpha_1) K(\mathbf{x}_1, \mathbf{x}_1) - y_2(\alpha_2^{\text{new,clipped}} - \alpha_2) K(\mathbf{x}_2, \mathbf{x}_1).$$

The updated  $b^{\text{new}}$  is then given by

$$b^{\text{new}} = \begin{cases} b_1 & \text{if } 0 < \alpha_1^{\text{new}} < C \\ b_2 & \text{if } 0 < \alpha_2^{\text{new,clipped}} < C . \\ \frac{b_1 + b_2}{2} & \text{otherwise} \end{cases}$$

④ **Repeat until convergence:** The final step of the algorithm is to check is there still any  $\alpha_i$  that violates the KKT conditions. If yes, then we will perform another SMO step. If not, then algorithm is then terminated, and output the desired  $\alpha$  and  $b$ . Compared to (2.4), often times we say the SMO converges when all KKT conditions are satisfied with small tolerance. That is, for a fixed  $\epsilon$ , we check for all  $\mathbf{x}_i$ , if

$$\begin{cases} \alpha_i = 0 & \implies y_i f(\mathbf{x}_i) \geq 1 - \epsilon \\ 0 < \alpha_i < C & \implies |y_i f(\mathbf{x}_i) - 1| \leq \epsilon . \\ \alpha_i = C & \implies y_i f(\mathbf{x}_i) \leq 1 + \epsilon \end{cases}$$

To summarize ①,②,③,④, we present the persudo code of the SMO algorithm as follow:

---

**Algorithm 1** Sequential Minimal Optimization (SMO)

---

**Require:** Training data  $\{(x_i, y_i)\}_{i=1}^n$ ,  $y_i \in \{-1, +1\}$ , kernel  $K$ , penalty  $C$

**Ensure:** Optimal dual variables  $\alpha, b$

- 1: Initialize  $\alpha_i \leftarrow 0$  for all  $i$ ,  $b \leftarrow 0$
  - 2: **while** KKT conditions are violated **do**
  - 3:   Select indices  $i \neq j$
  - 4:   Optimize  $(\alpha_i, \alpha_j)$  subject to  $0 \leq \alpha_i, \alpha_j \leq C, \alpha_i y_i + \alpha_j y_j = k$  using algorithm described in ②, ③.
  - 5:   Update threshold  $\alpha_i, \alpha_j, b$ .
  - 6: **end while**
  - 7: **return**  $(\alpha, b)$
- 

We now return to the wine data set in **R**. We implement SVMs using a Gaussian kernel to find the optimal decision boundaries among three classes. We choose 4 groups of binary covariates: (V1) and (V2); (V3) and (V6); (V4) and (V8); (V9) and (V10) (see Figure 1 for more detail about the covariates), all of which are tricky to classify using LDA or QDA. The data points are plotted in Figure 6, where different colors refer to different classes (cultivars). The SVM decision regions are shown in three different colors representing three cultivars.

To test the accuracy of the classifier  $f$ , we define the training error as

$$\mathbb{P}(f(x_i) \neq y_i) := \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{f(x_i) \neq y_i\}. \quad (2.8)$$

We may also use (2.8) to compute the training error on wine data set using different classification methods we discussed so far (LDA, QDA, SVM) and their performance is shown in Table 1. We can easily see that SVMs have the least training error compared to LDA and QDA. The choice of the kernel is also important, where Table 2 compares the training error using SVMs but with different kernels (linear SVM, polynomial kernel with degree 2, 3, sigmoid kernel, and Gaussian kernel).

|            | LDA    | QDA    | SVM with Gaussian kernel |
|------------|--------|--------|--------------------------|
| V1 and V2  | 0.1910 | 0.1853 | 0.1685                   |
| V3 and V6  | 0.2978 | 0.2752 | 0.2528                   |
| V4 and V8  | 0.4157 | 0.3876 | 0.3483                   |
| V9 and V10 | 0.1685 | 0.1404 | 0.1236                   |

Table 1: The table illustrates the training error for each classification method (LDA, QDA, SVMs with Gaussian Kernel) and each binary covariates.

|            | Linear    | Poly_d2   | Poly_d3   | Gaussian  | Sigmoid   |
|------------|-----------|-----------|-----------|-----------|-----------|
| V1 and V2  | 0.2134831 | 0.3876404 | 0.2078652 | 0.1685393 | 0.2303371 |
| V3 and V6  | 0.2977528 | 0.5393258 | 0.4157303 | 0.2528090 | 0.3651685 |
| V4 and V8  | 0.4044944 | 0.5561798 | 0.4887640 | 0.3483146 | 0.4719101 |
| V9 and V10 | 0.1348315 | 0.3539326 | 0.1516854 | 0.1235955 | 0.2808989 |

Table 2: The training error of the SVM using different kernels.

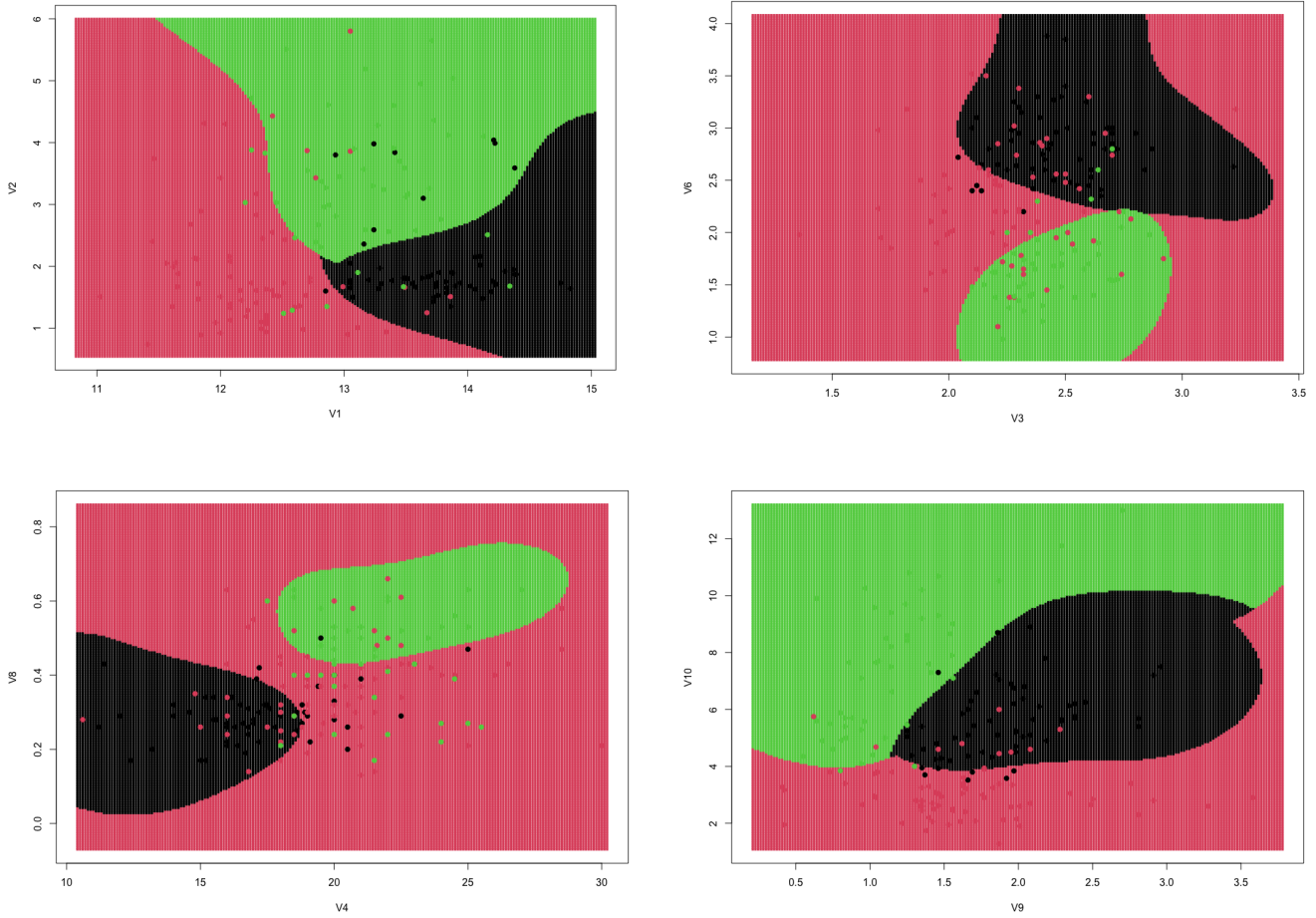


Figure 6: The decision regions using SVM with Gaussian kernel given by  $K(x, x') = \exp(-||x - x'||^2)$ .

## 2.2 $k$ th Nearest Neighbor (KNN)

## 2.3 Logistic Regression

Binary logistic regression is one of the easiest classification model to consider. We assume the response  $y_i$  is a Bernoulli random variable with  $\mathbb{P}(Y_i = 1|x_i) = p_i$ ,  $\mathbb{P}(Y_i = 0|x_i) = 1 - p_i$ . We further assume  $p_i$  is a function of the covariates  $x_i \in \mathbb{R}^p$  with coefficients vector  $\beta$ , then assuming independence, the likelihood function of the sample  $(x_1, y_1), \dots, (x_n, y_n)$  can be written as

$$L(y_1, \dots, y_n | \beta) = \prod_{i=1}^n p_i(\beta)^{y_i} \cdot [1 - p_i(\beta)]^{1-y_i}.$$

The first choice for  $p_i$  was the cumulative distribution function of a standard normal random variable. For each  $p_i \in [0, 1]$ , we can find a value  $\hat{z}$  such that

$$p_i = \Phi(\hat{z}) := \int_{-\infty}^{\hat{z}} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right) du,$$

and the function  $\Phi^{-1}(p_i)$  is called the *probit function*. However the probit model can be computationally challenging since it involves numerical methods to evaluate an integral, and the interpretation of parameters is not obvious [Rus26]. An alternative approach by Berkson is to use the following model:

$$p_i = \frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)}. \quad (2.9)$$

The function  $f(x) = e^x / (1 + e^x)$  is also known as the sigmoid function. Its graph is plotted in Figure 7

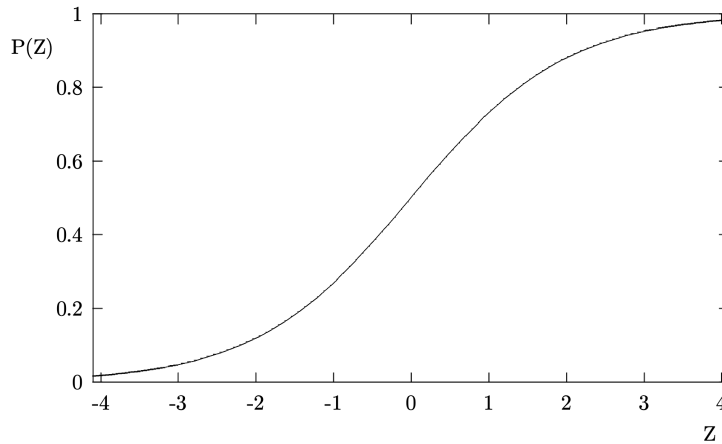


Figure 7: The graph of the sigmoid function  $f(x) = e^x / (1 + e^x)$ . The figure is taken from Cramer [Cra02]

The sigmoid function was invented in the 19th century, and was originally used to describe the growth of certain population under resource-limited environment, as well as the course of chemical reactions. It originates from the following differential equation:

$$\frac{dW(t)}{dt} = \beta W(t) \cdot (\Omega - W(t)), \quad (2.10)$$



where  $\beta, \Omega$  are constants. It is easy to show that the solution to (2.10) is given by

$$W(t) = \frac{\Omega e^{\beta t + C}}{1 + e^{\beta t + C}},$$

for some constant  $C$  that depends on the initial condition. A detailed history about logistic regression can be found in Cramer [Cra02]. Under the assumption given in (2.9), the log-likelihood function of the sample is given by

$$\ell(y_1, \dots, y_n | \beta) = \sum_{i=1}^n y_i \cdot \frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)} + (1 - y_i) \cdot \left[ 1 - \frac{\exp(x_i^\top \beta)}{1 + \exp(x_i^\top \beta)} \right],$$

and the optimal solution  $\hat{\beta} = (\hat{\beta}_0, \hat{\beta}_1, \dots, \hat{\beta}_{p-1})^\top$  can be obtained by solving

$$\frac{\partial \ell(y_1, \dots, y_n | \beta)}{\partial \beta_i} = 0.$$

Numerical methods are commonly used to obtain the zeros of the first derivative of  $\ell(y_1, \dots, y_n | \beta)$ , for example: Newton's method and gradient descent. After obtaining our estimate  $\hat{\beta}$ , the predicted logistic function is then

$$\hat{p}_i = \frac{\exp(x_i^\top \hat{\beta})}{1 + \exp(x_i^\top \hat{\beta})} \in [0, 1]. \quad (2.11)$$

A simulation is shown in Figure 8.

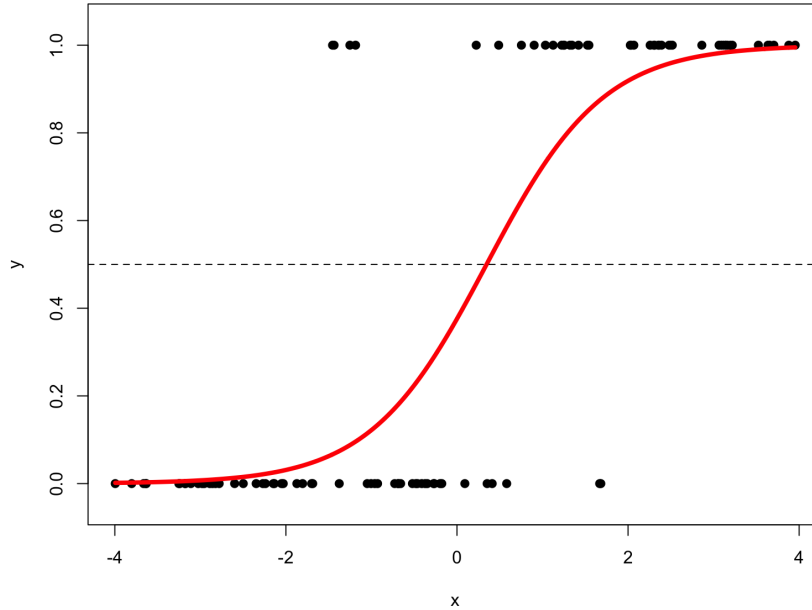


Figure 8: A simulation of a single covariate logistic regression. The response  $y_i = \{0, 1\}$  is binary and the model assumes  $p_i = \exp(\beta_0 + \beta_1 x) / (1 + \exp(\beta_0 + \beta_1 x))$ . The regression function is shown in red and the decision boundaries is the given by  $p = 0.5$ .

## 3 Tree-based Classification Algorithms

### 3.1 Decision Trees: An Introduction

Unlike many other classification algorithms that produce black-box models, decision trees offer transparency by representing the decision processes as a sequence of simple and intuitive rules (Mienye and Jere [MJ24]). The basic structure of a decision tree consists a bunch of nodes where decisions are made based on the attributes (or covariates) of the data. Each decision node has several children indicating different outcomes based on the attribute, and all the leaves nodes represent a set of classified data according to a sequence of decision-making processes. The idea is to recursively split the data into subsets based on attributes until some stopping criterion is met. The splitting algorithm first identifies the feature and threshold that leads to the best split based on specific criterion, then each children of the node represents a group of classified data according to the features determined by the root node. Each children will then be the root of a new tree and this process will continue recursively until some stopping criterion is met, typically when all the data inside each node is homogeneous, or the tree reaches a certain depth.

One crucial rule for constructing a decision tree is to define the splitting rules, i.e., how one should divide the data into subsets based on the attributes, since different splitting rules may lead to different tree structures. A deep decision tree has higher accuracy but is computationally expensive; a shallow decision tree is more computationally efficient but may be less accurate and may not fully capture the correlations and decision rules among attributes.

The following table consists of a dataset taken from Quinlan [Qui85], where we want to know whether a day is suitable for playing tennis based on the whether conditions. The attributes are 'Outlook' (sunny, overcast, rain), 'Temperature' (hot, mild, cool), 'Humidity' (high, normal), 'Wind' (strong weak), and the response of interest is simply 'Yes' or 'No'.

| Date | Outlook  | Temp | Humidity | Wind   | Play Tennis? |
|------|----------|------|----------|--------|--------------|
| 1    | Sunny    | Hot  | High     | Weak   | No           |
| 2    | Sunny    | Hot  | High     | Strong | No           |
| 3    | Overcast | Mild | High     | Weak   | Yes          |
| 4    | Rain     | Mild | High     | Weak   | Yes          |
| 5    | Rain     | Cool | Normal   | Weak   | Yes          |
| 6    | Rain     | Cool | Normal   | Strong | No           |
| 7    | Overcast | Cool | Normal   | Strong | Yes          |
| 8    | Sunny    | Mild | High     | Weak   | No           |
| 9    | Sunny    | Cool | Normal   | Weak   | Yes          |
| 10   | Rain     | Mild | Normal   | Weak   | Yes          |
| 11   | Sunny    | Mild | Normal   | Strong | Yes          |
| 12   | Overcast | Mild | High     | Strong | Yes          |
| 13   | Overcast | Hot  | Normal   | Weak   | Yes          |
| 14   | Rain     | Mild | High     | Strong | No           |

Table 3: The whether condition collected for 14 different days.

The figure on the next page illustrates two different decision trees of the data in Table 3 based on two different splitting rules and stopping criterion.

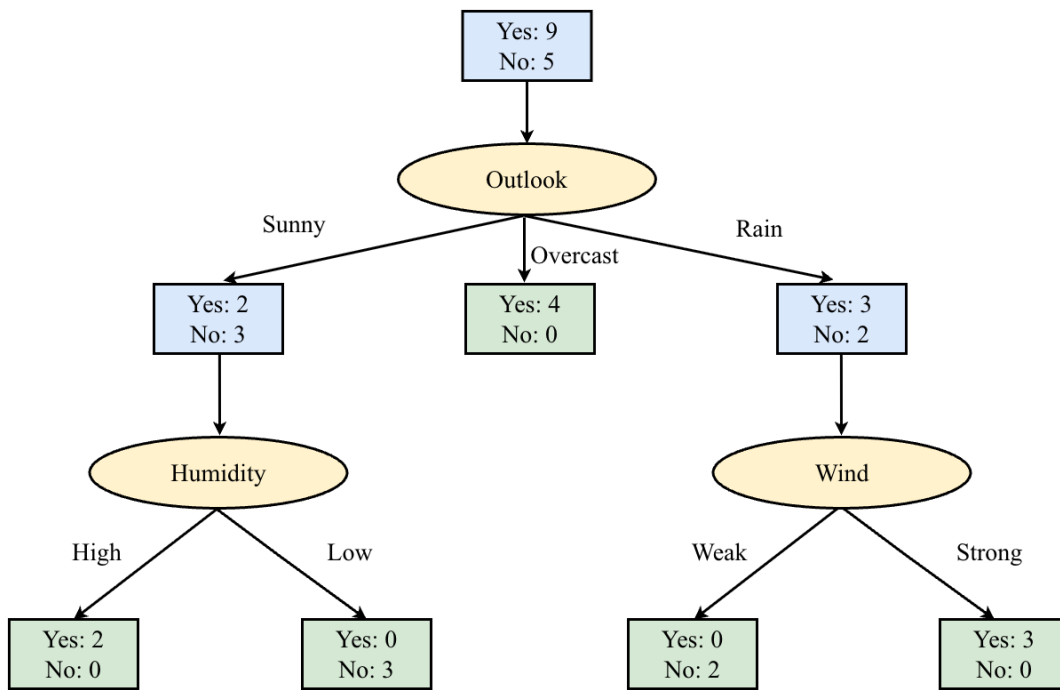


Figure 9: A decision tree constructed based on the whether data provided by Quilan. Each yellow node represents a decision node where current data are split according to the attributes, and each green node represents a leaf node where the data are homogeneous according to some decision rules.

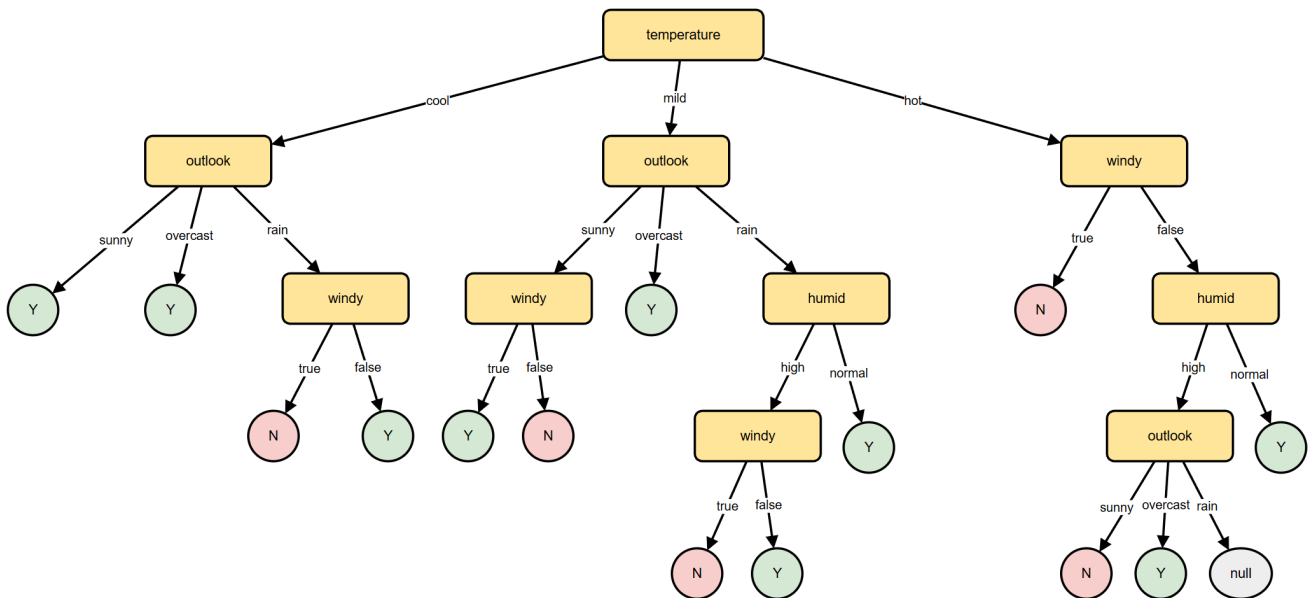


Figure 10: A more complicated decision tree structure for data given by Table 3. Each leaf corresponds to a unique date, and each internal node represents a decision making process based on some attributes.

From Figure 9 and 10, we can see that one advantage of decision tree is that the decision criteria is clear and easy to follow from the tree structure.

Several splitting rules had been proposed, and each rule led to a splitting algorithm. Namely the Information Gain (GI) used in Iterative Dichotomiser 3 (ID3) algorithm (Quinlan [Qui85]); Gini Index used in Classification and Regression Trees (CART) algorithm (Breiman et. al. [BFOS84]); the Chi-Square ( $\chi^2$ ) splitting criterion used in Chi-Squared Automatic Interaction Detection (CHAID) algorithm (Kass [Kas80]), etc.

### 3.2 Decision Tree Algorithms

**Information Gain (IG) and ID3 Algorithm [2]:** The idea comes from the notion of entropy used in information theory, which measures the unpredictability or randomness in a set of data. The entropy  $E$  of a set  $S$  with attribute  $A$  is defined by

$$E(S_A) = - \sum_{x \in S} p_x \log_2 p_x,$$

where  $x$  represents different target classes in  $S$ , and  $p_x$  is the proportion of the samples in  $S$  that belong to class  $x$ . For example, the entropy of Table 3 is given by

$$E(S_A) = - \left( \frac{9}{14} \times \log_2 \frac{9}{14} + \frac{5}{14} \times \log_2 \frac{5}{14} \right) = 0.9403,$$

since there are 9 ‘yes’s and 5 ‘no’s in the target class. The information gain (GI) for a split on  $S$  with attribute  $A$  is given by

$$IG(S, A) = E(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} E(S_v), \quad (3.1)$$

where  $\text{Values}(A)$  are different values that attribute  $A$  can take,  $S_v \subset S$  is the subset of data that has attribute  $v$ . Equation (3.1) computes the change in entropy from the original set  $S$  to  $S_v$  after the split, and a higher IG indicates a more effective attribute for splitting, since it results in more homogeneous subsets ([MJ24]). In Table 3, if we compute its IG based on attributes  $A = \text{outlook}$ ,  $B = \text{humidity}$ ,  $C = \text{temperature}$ ,  $D = \text{wind}$ , we see that

$$\begin{aligned} IG(S, A) &= 0.9403 - \left( \frac{5}{14} \cdot E(S_{A1}) + \frac{4}{14} \cdot E(S_{A2}) + \frac{5}{14} \cdot E(S_{A3}) \right) \\ &= 0.9403 - \left( \frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 \right) \\ &= 0.2467; \end{aligned} \quad (3.2)$$

$$\begin{aligned} IG(S, B) &= 0.9403 - \left( \frac{7}{14} \cdot E(S_{B1}) + \frac{7}{14} \cdot E(S_{B2}) \right) \\ &= 0.9403 - \left( \frac{7}{14} \cdot 0.9852 + \frac{7}{14} \cdot 0.5917 \right) \\ &= 0.1518; \end{aligned} \quad (3.3)$$

$$\begin{aligned} IG(S, C) &= 0.9403 - \left( \frac{3}{14} \cdot E(S_{C1}) + \frac{7}{14} \cdot E(S_{C2}) + \frac{4}{14} \cdot E(S_{C3}) \right) \\ &= 0.9403 - \left( \frac{3}{14} \cdot 0.9183 + \frac{7}{14} \cdot 0.8631 + \frac{4}{14} \cdot 0.8112 \right) \\ &= 0.0802; \end{aligned} \quad (3.4)$$

$$\begin{aligned}
IG(S, D) &= 0.9403 - \left( \frac{8}{14} \cdot E(S_{D1}) + \frac{6}{14} \cdot E(S_{D2}) \right) \\
&= 0.9403 - \left( \frac{8}{14} \cdot 0.8113 + \frac{6}{14} \cdot 1 \right) \\
&= 0.0481.
\end{aligned} \tag{3.5}$$

Hence based on our computation ((3.2); (3.3); (3.4); (3.5)),  $IG(S, A)$  (split based on outlook) maximizes the information gain, so Table 3 in fact shows the best split based on this criteria. The decision tree algorithm based on information gain is called Iterative Dichotomiser 3 (ID3) algorithm proposed by Quinlan ([Qui85]).

---

**Algorithm 2** ID3 Decision Tree Algorithm

---

**Require:** Training dataset  $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$

**Ensure:** Decision tree  $T$

```

1: function ID3( $\mathcal{D}$ )
2:   if  $\mathcal{D} = \emptyset$  then
3:     return a terminal node with default class  $c_{\text{default}}$ 
4:   end if
5:   if all instances in  $\mathcal{D}$  have the same class label  $y$  then
6:     return a terminal node with class  $y$ 
7:   end if
8:   if attribute set  $\mathcal{J}$  is empty then
9:     return a terminal node with the prevalent class in  $\mathcal{D}$ 
10:  end if
11:  Select the feature  $f$  that best splits the data using information gain
12:  Create a decision node for  $f$ 
13:  for each value  $b_i$  of feature  $f$  do
14:    Create a branch for  $b_i$ 
15:    Let  $\mathcal{D}_i = \{(x, y) \in \mathcal{D} : x_f = b_i\}$ 
16:    Recursively build the subtree  $T_i \leftarrow \text{ID3}(\mathcal{D}_i)$ 
17:    Attach  $T_i$  to the branch for  $b_i$ 
18:  end for
19:  return the decision node
20: end function

```

---

To summarize, the ID3 decision tree algorithm is a top-to-bottom, greedy search algorithm for constructing a decision tree. It begins with the entire dataset and iteratively divide the dataset based on the attribute that maximizes the information gain. The attribute that maximizes the information gain is then chosen as a decision node and partition the remaining dataset, and the process terminates when some stopping criteria is met. One disadvantage for ID3 decision tree algorithm is that it only works with discrete attributes, lacking the treatment for continuous cases. Also it is unable to prevent overfitting.

**Gini Index and CART Algorithm [3]:** The Gini index measures the probability of a randomly chosen sample being incorrectly classified if it was randomly labeled. It can also be used

to evaluate the split in the tree, and the Gini index for a set  $S$  is given by

$$\text{Gini}(S) = 1 - \sum_{i=1}^n p_i^2, \quad (3.6)$$

where  $n$  is the number of unique classes in the set, and  $p_i$  is the proportion of the samples in the set that belong to class  $i$ . The formula in (3.6) is equal to the probability that a randomly chosen sample  $x \in S$  with random class assignment  $\mathbb{P}(y = i) = p_i$  is misclassified. It is not hard to see that  $\text{Gini}(S) \in [0, 1]$ , and a smaller Gini index represents a more homogeneous set. By splitting the set  $S$  into two subsets  $S_1$  and  $S_2$ , we may compute the Gini index of the split by

$$\text{Gini}(S_1, S_2) = \frac{|S_1|}{|S|} \cdot \text{Gini}(S_1) + \frac{|S_2|}{|S|} \cdot \text{Gini}(S_2), \quad (3.7)$$

where a smaller  $\text{Gini}(S_1, S_2)$  would result in a better split. The Classification and Regression Tree (CART) algorithm proposed by Breiman et. al. [BFOS84] finds the best split based on minimizing the split Gini index defined in (3.7), where the idea is to create a binary decision tree such that at each decision node a question regarding the attributes is being asked (e.g., is the wind condition weak?). The answer is then simply ‘Yes’ or ‘No’, resulting in a partition of the data. The algorithm then applies iteratively to each set in the partition and the binary decision tree structure follows. The Table 4 below shows the Gini index for different splits on the root node of the tennis data.

| Attribute | Question  | $(Y, N)_{S_1}$ | $\text{Gini}(S_1)$ | $(Y, N)_{S_2}$ | $\text{Gini}(S_2)$ | $\text{Gini}(S_1, S_2)$ | $\Delta\text{Gini}(S)$ |
|-----------|-----------|----------------|--------------------|----------------|--------------------|-------------------------|------------------------|
| Outlook   | Overcast? | (4,0)          | 0.0000             | (5,5)          | 0.5000             | <b>0.3571</b>           | <b>0.1020</b>          |
| Outlook   | Rain?     | (3,2)          | 0.4800             | (6,3)          | 0.4444             | 0.4571                  | 0.0020                 |
| Outlook   | Sunny?    | (2,3)          | 0.4800             | (7,2)          | 0.3457             | 0.3937                  | 0.0655                 |
| Temp      | Cool?     | (3,1)          | 0.3750             | (6,4)          | 0.4800             | 0.4500                  | 0.0092                 |
| Temp      | Hot?      | (1,2)          | 0.4444             | (8,3)          | 0.3967             | 0.4069                  | 0.0523                 |
| Temp      | Mild?     | (5,2)          | 0.4082             | (4,3)          | 0.4898             | 0.4490                  | 0.0102                 |
| Humidity  | High?     | (3,4)          | 0.4898             | (6,1)          | 0.2449             | 0.3673                  | 0.0918                 |
| Humidity  | Normal?   | (6,1)          | 0.2449             | (3,4)          | 0.4898             | 0.3673                  | 0.0918                 |
| Wind      | Strong?   | (3,3)          | 0.5000             | (6,2)          | 0.3750             | 0.4286                  | 0.0306                 |
| Wind      | Weak?     | (6,2)          | 0.3750             | (3,3)          | 0.5000             | 0.4286                  | 0.0306                 |

Table 4: The computed Gini index for each potential split on root the node of the data given in 3. The Gini impurity reduction  $\Delta\text{Gini}(S)$  on the last column is defined as  $\Delta\text{Gini}(S) = \text{Gini}(S) - \text{Gini}(S_1, S_2)$ . The attribute ‘Outlook’ with question ‘Overcast’ has the smallest Gini split index.

Suppose at one step we have the data  $S := \{(x_1, y_1), \dots, (x_n, y_n)\}$ , a set of unique attributes  $f_1, \dots, f_m$ , and furthermore each attribute has several categories:  $f_i := \{f_{i1}, \dots, f_{ik(i)}\}$ . To find the best split, we first find all binary partitions of  $S$  that are induced by a single attribute taking one category and its complement:

$$S_{1ij} := \{x \in S : x(f_i) = j\}, \quad S_{2ij} = S \setminus S_{1ij}, \quad (3.8)$$

for all  $i \in [m], j \in [k(i)]$ . Then we choose the best partition  $(S'_1, S'_2)$  with the lowest value of split Gini index:

$$\text{Gini}(S'_1, S'_2) := \min_{i \in [m], j \in [k(i)]} \text{Gini}(S_{1ij}, S_{2ij}). \quad (3.9)$$

---

### Algorithm 3 CART Algorithm

---

**Require:** Training dataset  $\mathcal{S} = \{(x_1, y_1), \dots, (x_n, y_n)\}$

**Ensure:** Decision tree  $T$

```

1: function CART( $\mathcal{S}$ )
2:   if  $\mathcal{S} = \emptyset$  then
3:     return a terminal node with default class  $c_{\text{default}}$ 
4:   end if
5:   if all instances in  $\mathcal{S}$  have the same class label  $y$  then
6:     return a terminal node with class  $y$ 
7:   end if
8:   if attribute set  $\mathcal{J}$  is empty then
9:     return a terminal node with the prevalent class in  $\mathcal{S}$ 
10:  end if
11:  Partition the data set  $\mathcal{S}$  into  $\mathcal{S}_1, \mathcal{S}_2$  based on the criterion defined in (3.8) and (3.9) (If the
    attribute is continuous then use (3.10), (3.11)).
12:  Recursively build the subtree  $T_1 \leftarrow \text{CART}(\mathcal{S}_1)$  and  $T_2 \leftarrow \text{CART}(\mathcal{S}_2)$ .
13:  Attach  $T_1, T_2$  to the decision node.
14:  return the decision node
15: end function

```

---

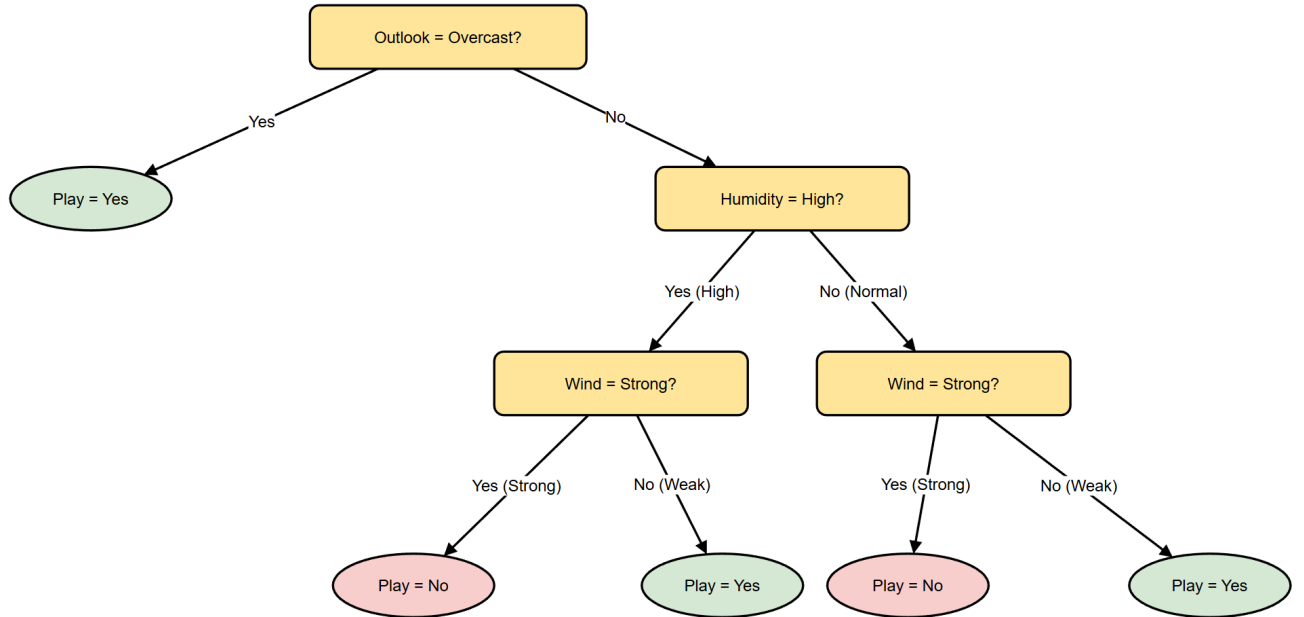


Figure 11: The decision tree of the data from 3 constructed using CART algorithm

Unlike ID3 algorithm, one advantage of CART algorithm is that it can also deal with continuous attributes. Since each node is simply a ‘Yes’ or ‘No’ question, we may partition the data into 2 sets based on the value of the attribute being greater or smaller than a chosen threshold. In this case, instead of the Gini index, we will use the sample variance as the measure of impurity. For a set  $S$  with continuous attribute  $Y$ , we define

$$\mathbf{Var}(S) = \frac{1}{|S|} \sum_{i=1}^m (Y_i - \bar{Y})^2, \quad (3.10)$$

where  $\bar{Y}$  is the sample mean. The variance of the split  $S = S_1 \cup S_2$  is then a weighted sum of the variance on each set:

$$\mathbf{Var}(S_1, S_2) = \frac{|S_1|}{|S|} \cdot \mathbf{Var}(S_1) + \frac{|S_2|}{|S|} \cdot \mathbf{Var}(S_2). \quad (3.11)$$

Then the CART algorithm will choose the split that has the minimum variance. The Figure 12 below shows the CART decision tree for the housing price based on longitude and latitude in California [Sha15]. The value in each grid represents the logarithm of the median housing price in that region.

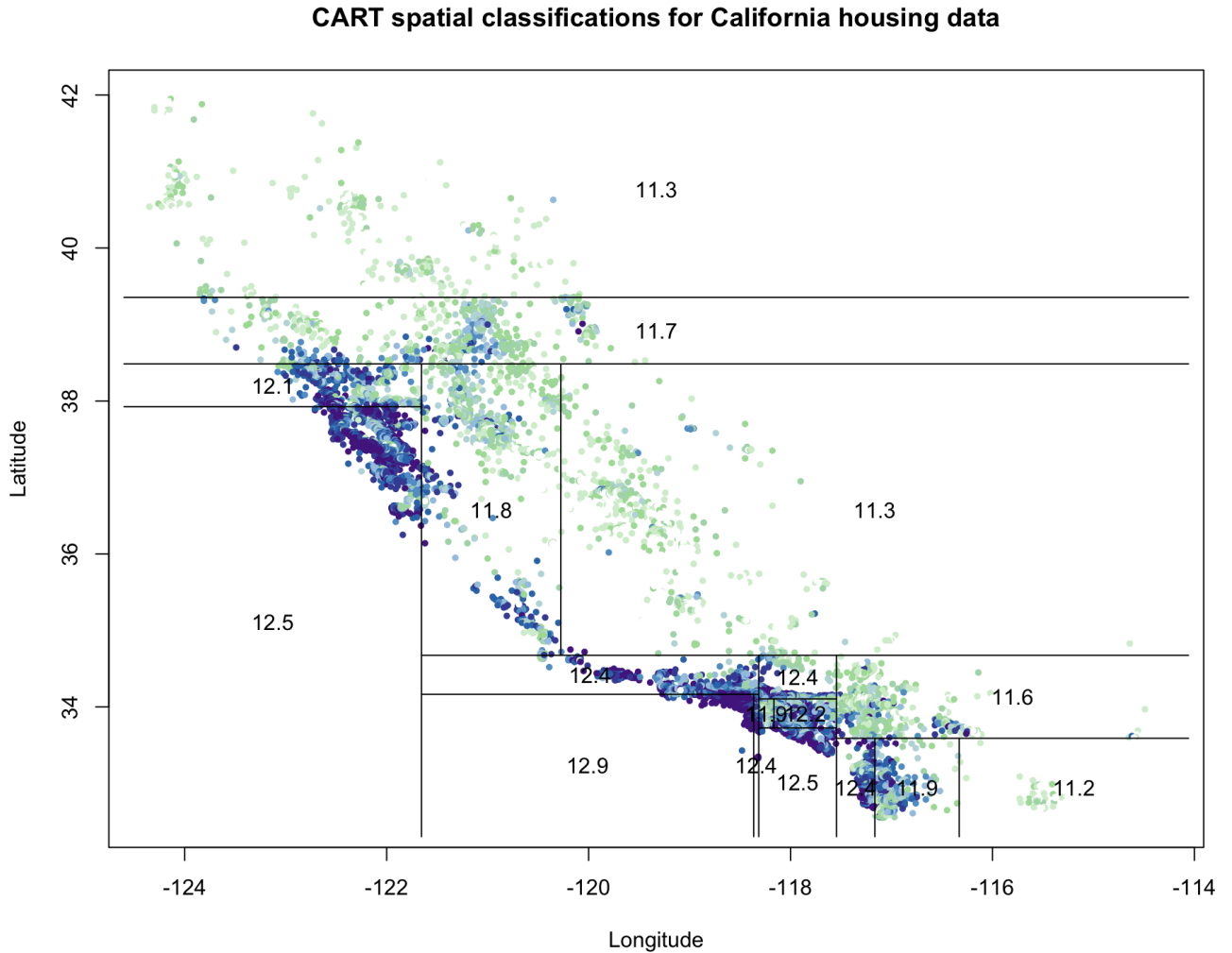


Figure 12: The CART decision tree for California housing price data.



### 3.3 Random Forests

## Part II

# Techniques for Analyzing Data

## 4 Dimension Reduction

Modern applications of statistical theory and methods may involve extreme datasets with huge number of measurements compared to relatively small sample sizes. High-dimensional data occurs when the number of covariates ( $p$ ) is greater than the number of observations ( $n$ ). In this section we briefly introduce two commonly used dimension reduction techniques: Fisher's discriminant analysis and principle component analysis.

### 4.1 Fisher's Discriminant Analysis

To achieve dimension reduction, our goal is to project our data  $x_i \in \mathbb{R}^p$  onto a smaller dimensional subspace  $\mathbb{R}^m$ ,  $m < p$  with the same label. Assume the smaller dimensional subspace can be spanned by vector  $v$ , and without loss of generality we assume  $\|v\| = 1$ . The problem now requires to find the optimal directional vector  $v$  such that the projections of  $x_i$  onto  $v$  can be well separated among different classes.

For simplicity, we first consider a dimensional reduction problem with two classes. The idea is to project each data point  $(x_i, y_i)$  onto the threshold function  $x(t) = t \cdot v$ . It can be shown that the projection of  $x_i$  onto the hyperplane is given by  $a_i := v^\top x_i$ , with the same label  $y_i$ .

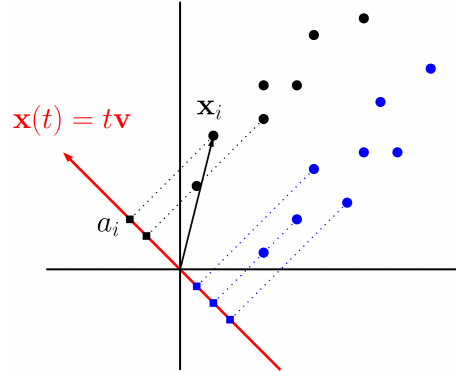


Figure 13: The geometric view of projecting each data point to the threshold function where each color represents a different class. The figure is taken from Chen [Che20]

Let  $\mathcal{C}_1, \mathcal{C}_2$  be two different classes,  $n_i$  be the number of data which belong to class  $i$  ( $i = 1, 2$ ). After projecting our data to the threshold function, we aim to measure the concentration of the data within the same class, and the separation of the data between different classes. We define the *with-class mean* by

$$\mu_1 = \frac{1}{n_1} \sum_{x_i \in \mathcal{C}_1} v^\top x_i = v^\top m_1, \quad \mu_2 = \frac{1}{n_2} \sum_{x_i \in \mathcal{C}_2} v^\top x_i = v^\top m_2, \quad (4.1)$$

and our first goal is to maximize  $|\mu_1 - \mu_2|$ . The larger the quantity is, the larger the separation between two classes occurs.

In the meantime we also would like to minimize the variance of data within each class, and we define the *in-class variance* by

$$s_1^2 = \sum_{x_i \in \mathcal{C}_1} (a_i - \mu_1)^2, \quad s_2^2 = \sum_{x_i \in \mathcal{C}_2} (a_i - \mu_2)^2, \quad (4.2)$$

and our second goal is to minimize both  $s_1^2$  and  $s_2^2$ . The smaller the quantity is, the larger the concentration within one class occurs. From (4.1), it follows that

$$\begin{aligned} (\mu_1 - \mu_2)^2 &= (\mathbf{v}^\top \mathbf{m}_1 - \mathbf{v}^\top \mathbf{m}_2) \cdot (\mathbf{v}^\top \mathbf{m}_1 - \mathbf{v}^\top \mathbf{m}_2) \\ &= \mathbf{v}^\top (\mathbf{m}_1 - \mathbf{m}_2) \cdot (\mathbf{m}_1 - \mathbf{m}_2)^\top \mathbf{v} \\ &:= \mathbf{v}^\top \mathbf{S}_b \mathbf{v}. \end{aligned}$$

The matrix  $\mathbf{S}_b := (\mathbf{m}_1 - \mathbf{m}_2) \cdot (\mathbf{m}_1 - \mathbf{m}_2)^\top \in \mathbb{R}^{p \times p}$  is called the *between-class scatter matrix*. It is clear that  $\mathbf{S}_b$  is symmetric and positive definite. From (4.2), for  $j = 1, 2$ , we also have

$$\begin{aligned} s_j^2 &= \sum_{x \in \mathcal{C}_j} (\mathbf{v}^\top \mathbf{x}_i - \mathbf{v}^\top \mathbf{m}_j)^2 \\ &= \mathbf{v}^\top \left[ \sum_{x_i \in \mathcal{C}_j} (\mathbf{x} - \mathbf{m}_j) \cdot (\mathbf{x}_i - \mathbf{m}_j)^\top \right] \mathbf{v} \\ &= \mathbf{v}^\top \mathbf{S}_j \mathbf{v}. \end{aligned} \quad (4.3)$$

The matrix  $\mathbf{S}_j := \sum_{x_i \in \mathcal{C}_j} (\mathbf{x} - \mathbf{m}_j) \cdot (\mathbf{x}_i - \mathbf{m}_j)^\top \in \mathbb{R}^{p \times p}$  is called the *within-class scatter matrix*. The *total within-class scatter matrix* is defined by  $\mathbf{S}_w = \mathbf{S}_1 + \mathbf{S}_2$ , which is also symmetric and positive definite. Hence

$$s_1^2 + s_2^2 = \mathbf{v}^\top (\mathbf{S}_1 + \mathbf{S}_2) \mathbf{v}. \quad (4.4)$$

To maximize  $|\mu_1 - \mu_2|$  and minimize  $s_1^2, s_2^2$ , we now introduce the optimization problem

$$\mathbf{v}^* := \arg \max_{\|\mathbf{v}\|=1} \frac{\mathbf{v}^\top \mathbf{S}_b \mathbf{v}}{\mathbf{v}^\top \mathbf{S}_w \mathbf{v}}. \quad (4.5)$$

**Theorem 1.** Suppose  $\mathbf{S}_w$  is invertible, then the maximizer  $\mathbf{v}^*$  of (4.5) is given by the eigenvector corresponds to the largest eigenvalue  $\mathbf{v}_1$  of  $\mathbf{S}_w^{-1} \mathbf{S}_b$ .

*Proof.* □

The idea for multi-classes  $\mathcal{C}_1, \dots, \mathcal{C}_k$  are the same. We wish to find  $k - 1$  directional vectors such that the projected mean onto any vector between any two classes are maximized. For each class  $j, j = 1, \dots, k$ , the *within-class scatter matrix* follows the same definition as before:

$$\mathbf{S}_j = \sum_{x_i \in \mathcal{C}_j} (\mathbf{x} - \mathbf{m}_j) \cdot (\mathbf{x} - \mathbf{m}_j)^\top, \quad \mathbf{m}_j = \frac{1}{n_j} \sum_{x_i \in \mathcal{C}_j} \mathbf{x}_i,$$

and the *total within-class scatter matrix* is simply

$$\mathbf{S}_w = \mathbf{S}_1 + \dots + \mathbf{S}_k.$$

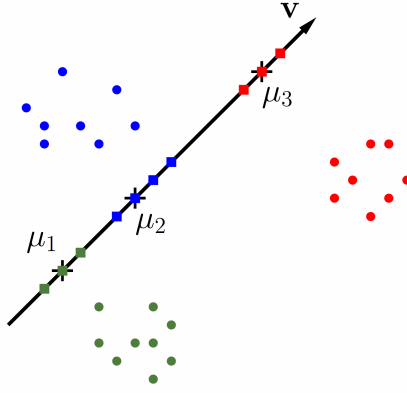


Figure 14: The geometric view of projections with three different classes.  $\mu_i$  represents the projected mean, and  $v$  is one of the two directional vectors. The figure is taken from Chen [Che20].

To measure the separability among different classes, we use the same definition for within-class mean as before: For  $j = 1, \dots, k$ ,

$$\mu_j = \frac{1}{n_j} \sum_{x_i \in \mathcal{C}_j} v^\top x_i := v^\top m_j.$$

The weighted average is defined by

$$\mu = \frac{1}{n_1 + \dots + n_k} \sum_{j=1}^k n_j \mu_j := v^\top m.$$

Finally the between-class scatter quantity is given by

$$\begin{aligned} \sum_{j=1}^k n_j (\mu_j - \mu)^2 &= \sum_{j=1}^k n_j (v^\top (m_j - m))^2 \\ &= v^\top \cdot \left[ \sum_{j=1}^k n_j (m_j - m) \cdot (m_j - m)^\top \right] \cdot v \\ &= v^\top S_b v, \end{aligned}$$

where  $S_b := \sum_{j=1}^k n_j (m_j - m) \cdot (m_j - m)^\top$  is the *between-class scatter matrix*. The optimization problem is eventually the same as (4.5):

$$v^* := \arg \max_{\|v\|=1} \frac{v^\top S_b v}{v^\top S_w v},$$

and the solutions are given by the eigenvectors with the largest eigenvalues. It suffices to solve

$$S_w^{-1} S_b v = \lambda v.$$

Note that when  $k = 2$ , we have

$$\mu = \frac{1}{n_1 + n_2} (n_1 \mu_1 + n_2 \mu_2), \quad m = \frac{1}{n} (n_1 m + n_2 m_2).$$

Despite  $S_w$  is invertible, it can be shown that  $\text{rank}(S_b) \leq k - 1$ , hence one can find at most  $k - 1$  eigenvectors as our solution. The figure below illustrates the three-class case in Figure 14:

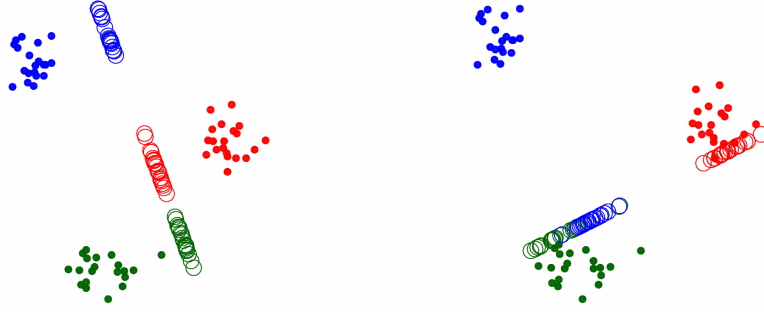


Figure 15: The multi-class case with  $k = 3$ . We find 2 eigenvectors as the solution, the threshold functions are simply the span for each eigenvector, as illustrated in the figure. The figure is taken from Chen [Che20].

We summarize the general procedure of linear discriminant analysis as the following algorithm:

---

**Algorithm 4** Multiclass LDA

---

**Require:** Design matrix  $X \in \mathbb{R}^{n \times p}$  with  $k$  classes  $\{\mathcal{C}_1, \dots, \mathcal{C}_k\}$ .

**Ensure:** At most  $k - 1$  threshold functions and projections of  $X$  onto them.

- 1: Compute the between-class and total within-class scatter matrix:

$$S_w = \sum_{j=1}^k \sum_{x_i \in \mathcal{C}_j} (x_i - m_j) \cdot (x_i - m_j)^\top, \quad S_b = \sum_{j=1}^k n_j (m_j - m) \cdot (m_j - m)^\top.$$

- 2: Solve the generalized eigenvalue problem  $S_b v = \lambda S_w v$  to find all non-zero eigenvectors  $V_m := [v_1, \dots, v_m]$  for some  $m \leq k - 1$ .
  - 3: Project the data  $X$  onto them:  $Y = X V_m \in \mathbb{R}^{n \times m}$ .
- 

## 4.2 Principle Component Analysis

## Part III

# References

## 5 Bibliography

- [BFOS84] L. Breiman, J. Friedman, R.A. Olshen, and C.J. Stone. Classification and regression trees. 1984.
- [Che20] Guangliang Chen. Linear discriminant analysis. *Math 253: Mathematical Methods for Data Visualization*, San José State University, 2020.
- [Cra02] J.S Cramer. The origins of logistic regression. 2002.
- [Kas80] G.V. Kass. An exploratory technique for investigating large quantities of categorical data. *Journal of the Royal Statistical Society.*, 29:119–127, 1980.
- [Mak00] Ginny Mak. The implementation of support vector machines using the sequential minimal optimization algorithm. 2000.
- [MJ24] Ibomoiye Domor Mienye and Nobert Jere. A survey of decision trees: Concepts, algorithms, and applications. *IEEE Access*, 12:86716–86727, 2024.
- [MRT12] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. 2012.
- [Pla98] John Platt. Sequential minimal optimization: A fast algorithm for training support vector machines. (MSR-TR-98-14), April 1998.
- [Qui85] J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1985.
- [Rus26] Russell. Course notes for math 523: Generalized linear models. 2026.
- [Sha15] Cosma Rohilla Shalizi. Classification and regression trees, chapter 13. 2015.